

Sets and Union-Find

EECS 281 study notes

How to read these notes. Anything inside an amber dashed box marked **NOT TESTED** is not required for this class. It is included because it is still useful to know. For this chapter that means **union-by-size and union-by-rank (section 13.4)**. Everything else is fair game.

13.1 Sets and Set Operations

13.1.1 Set Terminology

- A **set** is a well-defined collection of **distinct** elements (no duplicates).
- Main practical use: checking whether a specific value exists within a group of elements.

RUNNING EXAMPLE

$A = \{1, 2, 3\}$ $B = \{1, 3, 5\}$

Operation	Symbol	Definition	Result on A and B
Union	$\cup$	All elements in one set or the other	$A \cup B = \{1, 2, 3, 5\}$
Intersection	$\cap$	All elements that are members of both sets	$A \cap B = \{1, 3\}$
Set difference	$\setminus$ or $-$	Elements in one set but not the other	$A \setminus B = \{2\}$ $B \setminus A = \{5\}$
Symmetric difference	$\oplus$	Elements in either set but not both	$A \oplus B = \{2, 5\}$

13.1.2 Set Implementation

- The STL implements sets two ways (both covered in later chapters):
 - `std::unordered_set<>` uses **hash tables**
 - `std::set<>` uses **self-balancing trees**
- This chapter uses a simpler implementation: a **sorted vector** as the underlying container.
- Sorted vectors work well because **binary search** (chapter 15) checks membership in $\Theta(\log(n))$ time.

The set union algorithm on sorted vectors

1. Initialize an iterator at the beginning of each vector.
2. Compare the values at the two iterators.
3. Push the **smaller** value into the destination vector and increment that iterator.
4. If the two values are **equal**, push **one** copy and increment **both** iterators (avoids duplicates, which sets do not allow).
5. Repeat until one iterator reaches the end.
6. Then copy all remaining elements from the other vector into the destination.

WORKED EXAMPLE

Set A = {3, 4, 6, 8, 9, 10} Set B = {2, 4, 5, 8}

Step	A iter	B iter	Comparison	Action	A u B so far
1	3	2	$2 < 3$	push 2, advance B	2
2	3	4	$3 < 4$	push 3, advance A	2 3
3	4	4	equal	push 4, advance both	2 3 4
4	6	5	$5 < 6$	push 5, advance B	2 3 4 5
5	6	8	$6 < 8$	push 6, advance A	2 3 4 5 6
6	8	8	equal	push 8, advance both	2 3 4 5 6 8
7	9	end	B exhausted	copy rest of A (9, 10)	2 3 4 5 6 8 9 10

Why the tail copy is safe: both sets are traversed in sorted order, so whatever remains in the other set must be larger than everything already output. The result stays sorted and unique.

```
template <class InputIterator1, class InputIterator2, class OutputIterator, class Compare>
OutputIterator set_union(InputIterator1 first1, InputIterator1 last1,
                        InputIterator2 first2, InputIterator2 last2,
                        OutputIterator result, Compare comp) {
    // loop until one set reaches the end
    while (first1 != last1 && first2 != last2) {
        // *first1 < *first2: write it and advance first1
        if (comp(*first1, *first2)) { *result++ = *first1++; }
        // *first2 < *first1: write it and advance first2
        else if (comp(*first2, *first1)) { *result++ = *first2++; }
        // equal: write ONE copy, advance both
        else { *result++ = *first1++; ++first2; }
    } // while
    // copy whichever range still has elements left
    while (first1 != last1) { *result++ = *first1++; }
    while (first2 != last2) { *result++ = *first2++; }
    // iterator one past the end of the constructed range
    return result;
} // set_union()
```

comp(a, b) returns true if a is less than b.

TIME COMPLEXITY

Every iteration advances first1 or first2, so the algorithm touches $m + n$ elements total for sets of size m and n . All four set algorithms run in $\Theta(m + n)$.

All four operations use the same template

Only two things change: which branches write to the output, and whether the trailing copy loops are kept.

Function	Writes on $*first1 < *first2$	Writes on $*first2 < *first1$	Writes on equal	Keeps trailing copy loops?
set_union()	yes	yes	yes (one copy)	both
set_intersection()	no	no	yes (one copy)	neither
set_difference() (A \ B)	yes	no	no	only the one for range 1
set_symmetric_difference()	yes	yes	no	both

- **Intersection:** only the equal branch writes. The trailing loops are dropped, since once one set is exhausted nothing left in the other can be in the intersection.
- **Difference:** only the first branch writes (values in A that are not in B). Keep the loop that copies the rest of range 1.
- **Symmetric difference:** the two unequal branches write, the equal branch writes nothing. Keep both trailing loops.

```

// set_intersection(): only the equal case writes, no trailing loops
while (first1 != last1 && first2 != last2) {
    if (comp(*first1, *first2))    { ++first1; }
    else if (comp(*first2, *first1)) { ++first2; }
    else { *result++ = *first1++; ++first2; }
}
return result;

// set_difference(): only the first case writes, keep the range-1 tail loop
while (first1 != last1 && first2 != last2) {
    if (comp(*first1, *first2))    { *result++ = *first1++; }
    else if (comp(*first2, *first1)) { ++first2; }
    else { ++first1; ++first2; }
}
while (first1 != last1) { *result++ = *first1++; }
return result;

// set_symmetric_difference(): both unequal cases write, keep both tail loops
while (first1 != last1 && first2 != last2) {
    if (comp(*first1, *first2))    { *result++ = *first1++; }
    else if (comp(*first2, *first1)) { *result++ = *first2++; }
    else { ++first1; ++first2; }
}
while (first1 != last1) { *result++ = *first1++; }
while (first2 != last2) { *result++ = *first2++; }
return result;

```

13.1.3 Set Operations in the STL

- All four live in the `<algorithm>` library.
- Each takes: **two sorted iterator ranges**, an **output iterator** to the destination, and an **optional comparator** (operator< is used by default).
- Each returns an iterator **one past the last element** of the constructed range.
- **Requirement:** the input ranges must already be **sorted**. They do **not** need random access (input iterators are enough) and they do **not** need to be the same size.

Function	Constructs at result
<code>std::set_union</code>	the set union of the two sorted ranges [first1, last1) and [first2, last2)
<code>std::set_intersection</code>	the set intersection of the two sorted ranges
<code>std::set_difference</code>	the elements of [first1, last1) that are not in [first2, last2)
<code>std::set_symmetric_difference</code>	the symmetric difference of the two sorted ranges

```

std::vector<int32_t> v1 = {1, 2, 3};
std::vector<int32_t> v2 = {1, 3, 5};
std::vector<int32_t> v_union;
std::set_union(v1.begin(), v1.end(), v2.begin(), v2.end(), std::back_inserter(v_union));
for (int32_t val : v_union) {
    std::cout << val << " ";    // prints 1 2 3 5
}

```

13.2 Union-Find and Path Compression

13.2.1 Disjoint Sets and the Union-Find Data Structure

- Two sets are **disjoint** if they share no elements.
- Disjoint set problems: organize objects into groups and efficiently determine whether two objects are in the **same** group.

MOTIVATING EXAMPLE: COMPUTER NETWORK

Nodes are computers, edges are network connections. Two disjoint sets: {A, B, C, D, E} and {F, G, H, I, J, K}.

- Can C send data to E? **Yes**. Same disjoint set ($C \rightarrow B \rightarrow E$).
- Can C send data to I? **No**. Different disjoint sets, no path exists.
- Now add a connection between A and K. C to E is still yes. **C to I becomes yes**, because the two sets merged.
- Key point: neither C nor I had its own connections changed, yet their set membership changed. Membership is a property of the whole structure.

THE UNION-FIND DATA STRUCTURE

Also called a **disjoint-set data structure**. It tracks multiple disjoint sets and membership within them using two primary operations:

- `union_set(x, y)`: merge the set containing x with the set containing y .
- `find_set(x)`: identify which disjoint set x belongs to.

Naming note: `union` is a reserved C++ keyword so it cannot name a function, and `set_union` is already taken by the STL algorithm. Hence `union_set` and `find_set`.

Attempt 1: one sorted vector per disjoint set

- Store each disjoint set in its own sorted vector.
- `union_set()`: check if both elements are in the same vector; if not, merge all elements of one vector into the other. **Linear time**.
- `find_set()`: check every vector for the element. Worst case **linear time**.
- Both operations are linear, so this is inefficient, and it gets worse as the number of disjoint sets increases.
- **Fix:** store all elements in a **single** container instead of one container per set.

The representative system

- Each element remembers a **representative** that identifies which set it belongs to.
- The **ultimate representative** is the leader of the entire disjoint set. Its own representative is **itself**. This is how you recognize one.

CITIZENSHIP ANALOGY

A room full of people from many countries, all mingled. Ask each person who the leader of their country is. Everyone who names the US president is from the US, everyone who names the Canadian PM is from Canada, and so on. That answer groups them correctly.

Attempt 2: quick-find

- Every element stores its **ultimate** representative directly.
- `find_set()`: just look up the stored representative. **$\Theta(1)$** . To test if two elements are in the same set, compare their representatives.
- `union_set()`: must change the representative of **every** element in one of the sets, which is a linear traversal. **$\Theta(n)$** .

Element	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Rep	B	B	B	B	B	G	G	G	G	G	G	O	O	O	O

Quick-find: three sets led by B, G, and O.

Element	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Rep	B	B	B	B	B	B	B	B	B	B	B	O	O	O	O

After `union_set(A, K)`: six entries had to be rewritten. That is the $\Theta(n)$ cost.

Attempt 3: quick-union

- Drop the requirement that each element knows its **ultimate** representative.

- An element may store **any** element in its own disjoint set, as long as that element either
 1. directly knows the ultimate representative, or
 2. indirectly knows someone who does (follow the chain).
- Analogy: instead of naming the country's leader, name any other citizen of the same country. Following the chain (Alice → Bob → Cathy → US president) still identifies the country.
- `union_set(x, y)`: call `find_set()` on both to get the ultimate representatives; if different, point **one ultimate representative at the other**. Only **one** entry changes.

Element	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Rep	B	B	B	B	B	G	B	G	G	G	G	O	O	O	O

Quick-union after `union_set(A, K)`: only G's entry changed. F still says G, but G now says B, so F resolves to B.

THE COST OF QUICK-UNION

Find gets slower, because you may have to walk an entire chain. Worst case is a "**stick**":

Element	A	B	C	D	E	F	G	H
Rep	A	A	B	C	D	E	F	G

- `find_set(H)`: $H \rightarrow G \rightarrow F \rightarrow E \rightarrow D \rightarrow C \rightarrow B \rightarrow A$. A full $\Theta(n)$ traversal.
- `union_set()` calls `find_set()` twice, so union is also $\Theta(n)$ in the worst case (though faster on average in most situations).

13.2.2 Path Compression

- **Key insight:** the expensive $\Theta(n)$ traversal only needs to happen **once**. After learning that H's ultimate representative is A, store that fact.
- Even better: the traversal reveals the answer for **every element on the path**. If H's ultimate rep is A, and H pointed to G, then G's ultimate rep is A too, and so on back up the chain.
- **Path compression:** after a find, update the representative of **every element visited along the path** to point directly at the ultimate representative.
- This moves elements closer to their ultimate representative so future finds do less work.

Element	A	B	C	D	E	F	G	H
Before	A	A	B	C	D	E	F	G
After find(H)	A	A	A	A	A	A	A	A

Every node on the traversed path now points straight at A.

TREE VIEW

Person A is the ultimate representative. B points to A, C points to B, D points to C. Calling `find_set(D)` walks $D \rightarrow C \rightarrow B \rightarrow A$. Since B, C, and D all turn out to share ultimate representative A, path compression re-points **all three** directly at A, flattening that branch of the tree.

13.3 Implementing a Union-Find Container

Storage

- A single private vector `reps` of indices. **The value at index i is the index of element i 's representative.**
- Initialization: every element starts in its own disjoint set of size 1, so `reps[i] = i`. Do **not** zero-initialize, which would wrongly claim element 0 represents everyone.

index	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	...	[n-1]
reps	0	1	2	3	4	5	6	7	...	n-1

Initial state: every element represents itself.

```

class UnionFind {
    // value at index i is the index of the ith element's representative
    std::vector<int32_t> reps;
public:
    UnionFind(int32_t size) {
        reps.reserve(size);
        for (int32_t i = 0; i < size; ++i) {
            reps.push_back(i);    // at the beginning, every node represents itself
        } // for i
    } // UnionFind()
};

// the loop can be condensed to one line:
std::iota(reps.begin(), reps.end(), 0);

```

Walkthrough with five elements

Operation	reps after (index 0..4)	Explanation
start	0 1 2 3 4	five separate sets
union_set(3, 4)	0 1 2 3 3	set 4's rep to 3 (the choice of which side to update is arbitrary here)
union_set(2, 3)	0 1 2 2 3	set 3's rep to 2. Now {2, 3, 4} is one set with ultimate rep 2
union_set(1, 2)	0 1 1 2 3	set 2's rep to 1. Chain is now 4 → 3 → 2 → 1
find_set(4)	0 1 1 1 1	walk 4 → 3 → 2 → 1; 1 represents itself so it is the ultimate rep. Path compression re-points 2, 3, 4 to 1
union_set(0, 4)	0 0 1 1 1	update the rep of 4's ultimate representative (which is 1), not 4 itself

COMMON MISTAKE

For union_set(0, 4), do **not** just set reps[4] = 0. That would give 0 1 1 1 0, which pulls 4 out of the set containing 1, 2, and 3. An element must never be moved out of its original set by a union. Always update the **ultimate representative** of one side.

find_set()

- **Pass 1:** loop up the chain of representatives until you reach an element that represents itself.
- **Pass 2:** path compression, set every element along that path to the ultimate representative.

```

// find_set(x) returns the ultimate representative of x
int32_t find_set(int32_t x) {
    int32_t current = x;
    // pass 1: find the ultimate representative
    while (reps[x] != x) {
        x = reps[x];
    } // while
    // x now stores the ultimate representative
    // pass 2: path compression, set all reps along path to x
    while (reps[current] != x) {
        int32_t next = reps[current];
        reps[current] = x;
        current = next;
    } // while
    return x;
} // find_set()

```

Recursive version, much shorter:

```
int32_t find_set(int32_t x) {
    if (x == reps[x]) {
        return x;
    } // if
    reps[x] = find_set(reps[x]); // compression happens on the way back up
    return reps[x];
} // find_set()
```

union_set()

```
// union_set() unions the elements x and y into the same set
void union_set(int32_t x, int32_t y) {
    int32_t x_rep = find_set(x);
    int32_t y_rep = find_set(y);
    reps[y_rep] = x_rep;
} // union_set()
```

- This arbitrarily always updates **y**'s representative. It works, but it is not ideal. Section 13.4 covers smarter choices.

Example 13.1 earliest_acquaintance()

PROBLEM

n students with IDs 0 to $n-1$ on a social media site. You are given `num_students` and a vector of logs:

```
struct Log {
    int32_t timestamp; // YYYYMMDD, so integers compare correctly as dates
    int32_t id_A;
    int32_t id_B;
};
```

Each log records when `id_A` and `id_B` became friends. Friendship is symmetric. Person A is **acquainted** with person B if they are in the same friend group (friends directly, or friends of someone acquainted with B).

Return the earliest timestamp at which **every** person is acquainted with every other person, or -1 if that never happens.

Example 1. `num_students = 5`, `logs = {20200229, 2, 3}, {20200227, 1, 4}, {20200303, 0, 3}, {20200228, 0, 4}, {20200301, 1, 2}, {20200226, 0, 2}`. Answer: **20200229**.

Date	New friendship	Disjoint sets after
before	none	{0} {1} {2} {3} {4}
2/26	0 and 2	{0,2} {1} {3} {4}
2/27	1 and 4	{0,2} {1,4} {3}
2/28	0 and 4	{0,1,2,4} {3}
2/29	2 and 3	{0,1,2,3,4} ← one set left, return 20200229

On the 29th, person 0 is directly friends with 2 and 4, acquainted with 1 through 4, and acquainted with 3 through 2.

Example 2. `num_students = 5`, `logs = {20200304, 1, 4}, {20200307, 0, 2}, {20200306, 3, 4}`. Answer: **-1**.

Date	New friendship	Disjoint sets after
3/4	1 and 4	{1,4} {0} {2} {3}
3/6	3 and 4	{1,3,4} {0} {2}
3/7	0 and 2	{1,3,4} {0,2} ← logs exhausted, still 2 sets, return -1

KEY INSIGHT

Two students are acquainted **if and only if** they are in the same disjoint set. That makes this a union-find problem.

Algorithm

1. **Sort** the vector of logs by timestamp, so you traverse from earliest to latest.
2. **Initialize** a union-find container with each student as its own representative.

3. **Loop** over the logs. For each new friendship, union the two students together.
4. If only one disjoint set remains, return that timestamp. If all logs are traversed with multiple sets left, return -1 .

HOW TO DETECT "ONE SET LEFT" EFFICIENTLY

- **Naive:** after each friendship, scan the container counting elements whose rep equals themselves. That is $\Theta(n)$ per check, too slow.
- **Better:** count the number of times two students in **different** sets are unioned. Each such union reduces the number of disjoint sets by exactly one. Starting from n sets, you reach one set after exactly $n - 1$ such unions.
- Detect "different sets" by calling `find_set()` on both and comparing the ultimate representatives.
- This makes the check **$\Theta(1)$** : just increment or decrement a counter.

Two equivalent counters: start at 0 and increment until it reaches $n - 1$ (counts friendships between unacquainted students), or start at n and decrement until it reaches 1 (counts remaining disjoint sets).

```
struct LogComp {
    bool operator() (const Log& lhs, const Log& rhs) const {
        return lhs.timestamp < rhs.timestamp;
    }
};

// find the representative (using path compression)
int32_t find(std::vector<int32_t>& reps, int32_t id) {
    if (id == reps[id]) { return id; }
    reps[id] = find(reps, reps[id]);
    return reps[id];
} // find()

int32_t earliest_acquaintance(std::vector<Log>& friendships, int32_t num_students) {
    std::sort(friendships.begin(), friendships.end(), LogComp()); // can also use a lambda
    std::vector<int32_t> reps(num_students);
    int32_t count = 0; // can also start at n and decrement
    for (int32_t i = 0; i < num_students; ++i) {
        reps[i] = i; // or use std::iota()
    } // for i
    for (const Log& f : friendships) {
        int32_t rep_A = find(reps, f.id_A);
        int32_t rep_B = find(reps, f.id_B);
        // different reps means different disjoint sets, so this union merges two groups
        if (rep_A != rep_B) {
            reps[rep_B] = rep_A;
            ++count;
        } // if
        // done when n - 1 connections are made (check count == 1 if decrementing from n)
        if (count == num_students - 1) {
            return f.timestamp;
        } // if
    } // for f
    return -1; // fewer than n - 1 merging unions were made
} // earliest_acquaintance()
```

NOT TESTED

13.4 Union-by-Size and Union-by-Rank

Why another optimization is needed

- The union above blindly points the second set's ultimate rep at the first's. That still allows the worst case.
- Start with 0 1 2 3 4 all separate, then call `union_set(3,4)`, `union_set(2,3)`, `union_set(1,2)`, `union_set(0,1)`. Result: `reps = 0 0 1 2 3`, a stick. Both union and find become $\Theta(n)$.
- **Path compression does not fix this.** It improves the **amortized** cost of union and find, but not the **worst-case** cost, because you still have to do that $\Theta(n)$ traversal **at least once**, and nothing stops the stick from forming in the first place.

13.4.1 Union-by-Size

- Additionally track the **size** of each disjoint set.
- On union, always update the representative of the **smaller** set. Ties broken arbitrarily.

Operation	Sizes involved	reps after (0..4)
start	all 1	0 1 2 3 4
union_set(3,4)	1 vs 1, tie	0 1 2 3 3
union_set(2,3)	1 vs 2, so 2 is updated	0 1 3 3 3
union_set(1,2)	1 vs 3, so 1 is updated	0 3 3 3 3
union_set(0,1)	1 vs 4, so 0 is updated	3 3 3 3 3

- No stick forms. Worst-case cost of a single union or find drops from $\Theta(n)$ to $\Theta(\log(n))$.
- Reason: union-by-size guarantees the longest representative chain is at most $\Theta(\log(n))$ long.

13.4.2 Union-by-Rank

- Track the approximate **height (rank)** of each disjoint set instead of its size.
- Rank examples: a single node has rank 0; a root with only direct children has rank 1; a two-level tree has rank 2; and so on.
- On union, always update the representative of the set with the **smaller rank**. The larger rank stays unchanged.
- If the two ranks are **equal**, it does not matter which side is updated, and the resulting rank is **incremented by one**.

RANK IS ONLY AN APPROXIMATION

Rank approximates the longest chain from an element to its ultimate representative, but it is **not recomputed** when path compression shortens the real chain. Example: a rank-3 set stays labeled rank 3 even after `find_set(T)` flattens T and S directly onto the root N.

13.4.3 Union-Find Complexity Analysis

For $n-1$ union operations and m find operations:

Optimizations used	Total complexity
Path compression only	$O(n + m \log(n))$ when $m < n$; $O(m \log(n))$ when $m \geq n$
Union-by-rank or union-by-size only	$O(m \log(n))$, since find and union are both bounded by the largest rank or size, which is itself $O(\log(n))$
Path compression + union-by-size or union-by-rank	$\Theta(m \alpha(n))$ for any m operations, so amortized $\Theta(\alpha(n))$ per find and per union

- $\alpha(n)$ is the **inverse Ackermann function**. It grows so slowly that for any reasonable n its value can be treated as a constant.
- So with both optimizations, `find_set()` and `union_set()` are effectively **amortized constant time**.
- Source: Tarjan and van Leeuwen, "Worst-Case Analysis of Set Union Algorithms" (1984), and chapter 21 of CLRS (3rd ed.).

Complexity Summary

Implementation	find_set() worst case	union_set() worst case	Notes
One sorted vector per set	$\Theta(n)$	$\Theta(n)$	must scan every vector / merge two vectors
Quick-find (store ultimate rep)	$\Theta(1)$	$\Theta(n)$	union must rewrite every element of one set
Quick-union (store any rep)	$\Theta(n)$	$\Theta(n)$	union is $\Theta(n)$ only because it calls find twice

Quick-union + path compression	$\Theta(n)$	$\Theta(n)$	worst case unchanged, but amortized cost improves
+ union-by-size or by-rank NOT TESTED	$\Theta(\log(n))$	$\Theta(\log(n))$	chain length capped at $\Theta(\log(n))$
+ both NOT TESTED	—	—	amortized $\Theta(\alpha(n))$, essentially constant

Quick Reference: Facts Worth Memorizing

- **Counting disjoint sets:** count the elements whose representative equals themselves. Each one is an ultimate representative of exactly one set.
- **Merging all sets:** if there are k disjoint sets, you need at least $k - 1$ union operations to reduce them to one.
- **What a find changes:** with path compression, `find_set(j)` re-points every element along the chain from j to its ultimate representative. Trace the chain fully before answering questions about it.
- **When a find changes nothing:** if the element's representative is already its ultimate representative (that is, no intermediaries on the path).
- **Max elements changed by one find:** (size of the largest disjoint set) $- 1$, achieved by a stick where you call `find` on the farthest element. The ultimate representative itself never changes.
- **Largest possible set:** with N elements and r ultimate representatives, there are r disjoint sets; the biggest can hold $N - (r - 1)$ elements, since the other $r - 1$ sets need at least one element each.
- **Union on a fresh container:** if every element is its own representative, one `union_set()` merges two singletons and changes exactly **one** representative.
- **Set algorithms on sorted vectors:** finding elements in one vector but not the other is $\Theta(n)$ if the vectors are already sorted, and $\Theta(n \log(n))$ if you must sort them first (which still beats the $\Theta(n^2)$ brute force of scanning one vector for each element of the other).

WHERE UNION-FIND SHOWS UP

- Image processing (identifying and updating regions of similar color)
- Networking (which servers can communicate with each other)
- Social networks (grouping people by shared interests or friend groups)
- Counting **connected components** in a graph
- **Kruskal's algorithm** for minimum spanning trees, revisited in chapter 20
- Constraint satisfaction, for example checking whether a set of $x==y$ and $x!=y$ equations is satisfiable: union all the equality pairs first, then verify that every inequality pair lands in different disjoint sets